

Practical 4

AIM : Feed Forward Backpropagation Neural Network

1. Implement the Feed Forward Backpropagation algorithm to train a neural network.
2. Use a given dataset to train the neural network for a specific task.
3. Evaluate the performance of the trained network on test data.

CODE :

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()

X = data.data
y = data.target

print("Dataset Shape:", X.shape)
print("Number of Features:", X.shape[1])
print("Classes:", data.target_names)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Feed Forward Backpropagation Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(10, 5),
    activation="relu",
    solver="adam",
```

Name : Shubham Sanjay Karape

Roll No : T085

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE

SUBJECT : ARTIFICIAL INTELLIGENCE

```
learning_rate_init=0.001,
max_iter=1000,
random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nFeed Forward Backpropagation Neural Network")
print("-----")

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy * 100, "%")

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=data.target_names
    )
)

# Training Loss Graph
plt.figure(figsize=(10, 6))

plt.plot(
    model.loss_curve_,
    linewidth=2
)

plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

Name : Shubham Sanjay Karape

Roll No : T085

SUBJECT : ARTIFICIAL INTELLIGECE

```
PS C:\Users\DELL\OneDrive\Desktop\AI Practicals> python Practical4.py
Dataset Shape: (178, 13)
Number of Features: 13
Classes: ['class_0' 'class_1' 'class_2']

Feed Forward Backpropagation Neural Network
-----

Training Samples: 142
Testing Samples: 36

Actual Values:
[0 2 0 1 1 0 0 1 1 2 1 2 0 2 0 1 1 0 1 1 0 0 1 1 0 2 1 2 0 2 1 2 2 2]

Predicted Values:
[0 1 0 1 1 0 0 1 1 2 1 2 0 2 0 1 1 0 1 1 0 0 1 1 0 2 1 2 0 2 1 2 2 2]

Accuracy: 97.2222222222221 %

Classification Report:
      precision    recall  f1-score   support

   class_0       1.00      1.00      1.00        12
   class_1       0.93      1.00      0.97        14
   class_2       1.00      0.90      0.95        10

   accuracy              0.97        36
  macro avg       0.98      0.97      0.97        36
 weighted avg       0.97      0.97      0.97        36
```

The screenshot shows a software window titled "Figure 1" containing a line plot titled "Training Loss Curve". The x-axis is labeled "Iterations" and ranges from 0 to 500 with major grid lines every 100 units. The y-axis is labeled "Loss" and ranges from 0.0 to 1.0 with major grid lines every 0.2 units. A blue line represents the training loss, starting at approximately (0, 1.1) and decreasing rapidly at first, then more slowly, reaching a value of about 0.03 at 550 iterations. A tooltip in the top right corner of the plot area displays the text "Favorites (Ctrl+Shift+O)". At the bottom of the window, a status bar shows the coordinates "(x, y) = (387.4, 0.830)". The Windows taskbar is visible at the very bottom, showing the search bar, task view button, and several application icons.

Name : Shubham Sanjay Karape
Roll No : T085